Symbols

:clojure.spec.alpha/invalid: keyword 484

:shrunk key 491

:smallest [(0)] key 491

./gradlew command 377

./gradlew tasks meta-task 377

.gradle.kts extension 378

(.) macro 332

(agent) function 569

(alter) form 568

(apply) form 319

(cast) name 108

(concat) form 534

(conj <coll> <elt>) function 328

(conj) operation 557

(cons <elt> <coll>) function 328

(cons) operation 557

(constantly) function 330

(def <symbol> <value?>) special form 310

(def) form 310

(defmacro) special form 338

(defn) macro 305

(deref) form 311

(do <expr>*) special form 310

(do) form 309

(dosync) form 568

(every? <pred-fn> <coll>) function 328

(filter) call 535

(filter) form 323

(first <coll>) function 328

(fn <name>? [<arg>*] <expr>*) special form 310

(for) form 531

(future-done?) nonblocking call 564

(if <test> <then> <else>) special form 310

(if) special form 338

(import) form 332

(lazy-seq) form 534

(lazy-seq) macro 534

(let [<binding>*] <expr>*) special form 310

(let) form 317

(loop) form 320

(macroexpand-1) form 339

(macroexpand) form 339

(map) function 314

(new) form 332

(nth) function 313

(partial) form 535

(pcalls) function 564

(println) function 308

(proxy) form 334

(proxy) macro 333

(quote <form>) special form 310

(quote) form 311

(recur) form 320

(reduce) form 324

(ref) form 311, 565, 568

(rest <coll>) function 328

(send) call 570

(seq <coll>) function 328

(seq? <o>) function 328

(sort-by) new form 319

(take) form 534

(unless) form 336

(unless) function 337

(unless) macro 339

(var <symbol>) special form 310

(var) special form 311

(vec) form 313

(vector) form 313

(wait-and-log) call 570

@Container annotation 470

@file:JvmName("AlternateClassName") annotation 295

@FunctionalInterface annotation 146, 202

@JvmOverloads annotation 296

@Mojo annotation 373

@NotNull annotation 288

@Nullable annotation 288

@Test annotation 460

@Testcontainers annotation 469

&env special varaible 339

&form special variable 339

#() form 323 – 324

#define preprocessor directive 336

#include preprocessor directive 336

A

AbstractQueuedSynchronizer class 173

access control 33 – 34, 588

accessor methods 236

Account class 601

Account interface 601

AccountManager class 190

accuracy 221 – 222

aconst_init opcode 631

add opcode 108

- -add-exports switch 41, 43

- -add-host flag 418

- -add-modules switch 42

- -add-opens command-line option 45

Add-Opens JAR manifest attribute 45

- -add-opens switch 42

- -add-reads switch 41

add() method 186

additional static arguments 590

agents 569 – 570

aggregator modules 39

algebraic data types 73

Algorithm Design Manual, The (Skiena) 128

all function 279

allocator object 617

aload_0 opcode 102

alternative JVM languages

how to choose 262 – 265

developers using language 264 – 265

difficulty of learning language 264

language interoperating well with Java 263 – 264

project area low-risk 262 – 263

tooling and test support for language 264

JVM supporting 265 – 268

compiler fictions 267 – 268

performance 266

runtime environments for non-Java languages 266 – 267

language zoology 252 – 256

dynamic vs. static typing 253 – 254

imperative vs. functional languages 254 – 255

interpreted vs. compiled languages 252 – 253

reimplementation vs. original 255 – 256

polyglot programming on JVM 256 – 262

contenders 260 – 262

non-Java language, reasons for using 257 – 259

up-and-coming languages 259 – 260

Amdahl's law 121 – 122

andThen() method 513

anonymous classes 607

any function 279

AOT (ahead-of-time) compiled languages 234

APIs

Foreign Function and Memory API 612 – 618

handling native memory in Panama 617 – 618

LibSPNG example 614 – 617

replacing Unsafe with supported 604 – 608

hidden classes 607 – 608

VarHandles 605 – 607

using BlockingQueue APIs 194 – 195

APM (application performance monitoring) 432

AppClassLoader class loader 86

application modules 38 – 39

application plugin 381

applications

developing Java applications with Docker 411 – 424

building image with Gradle 412 – 414

debugging in Docker 421 – 423

local development with Docker Compose 418 – 421

logging with Docker 423 – 424

ports and hosts 416 – 418

running build in Docker 414 – 416

selecting base image 411 – 412

Gradle making 380 – 381

apply() method 513

architecting for modules 46 – 52

Java 8 Compact Profiles 47 – 49

multirelease JARs 49 – 52

split packages 47

arithmetic

Clojure 315 – 316

opcodes 107

arity of functions 329 – 330

AS Docker command 414

assemble task 388

asSequence() function 529

assertj library 362

associateBy function 279

associateWith function 279

AST (abstract syntax tree) 308

astore opcode 107

async function 556

async/await approach 624

atomic classes 170 – 172

AtomicBoolean class 170

AtomicInteger class 170

AtomicLong class 170

AtomicReference class 170

Attach API 42

automatic instrumentation 213 – 214

automatic modules 39 – 40

automating

operations 346

static analysis 389 – 390

await() method 174

B

back pressure 189

backward compatibility 640 – 641

BadInit class 90

bare metal machines 402

barriers 174

base plugin 378

bean style 145

Beck, Kent 441, 443

BiFunction interface 513

binding 301

Bjarnason, Rúnar 516

block-structured concurrency (pre-Java 5) 128 – 147

deadlocks 132 – 135

fully synchronized objects 131 – 132

immunitibility 144 – 147

state model for threads 130 – 131

synchronization

locks and 128 – 130

reasons for 135 – 137

thread states and methods 137 – 143

deprecated thread methods 143

interrupting threads 140 – 142

working with exceptions and threads 142 – 143

volatile keyword 137

BLOCKED state 131

BlockingQueue interface 188, 194

BlockingQueue pattern 188 – 197

using BlockingQueue APIs 194 – 195

using WorkUnit 196 – 197

boolean parameter 159

BootstrapClassLoader class loader 86

brackets, in Clojure 308 – 309

build life cycle 350

<build><plugins> section 359

build tools

Gradle 376 – 400

adding Kotlin 387 – 388

automating static analysis 389 – 390

building 379 – 381

customizing 397 – 400

dependencies in 382 – 387

installing 376 – 377

moving beyond Java 8 390

scripts 378

tasks 377 – 378

testing 388 – 389

using plugins 378 – 379

using with modules 391 – 397

work avoidance 381 – 382

importance of for developers 345 – 350

automating tedious operations 346

ensuring consistency between developers 349 – 350

managing dependencies 346 – 349

Maven 350 – 376

adding another language 355 – 357

authoring Maven plugins 372 – 376

build lifecycle 350 – 351

building 353 – 354

controlling manifest 354

dependency management 360 – 364

installling/POM 351 – 353

modules and 369 – 372

moving beyond Java 8 365 – 366

multirelease JARs in 366 – 369

reviewing 364 – 365

testing 357 – 360

- -build-cache command-line flag 382

Builder class 145

building modular apps 40 – 46

command-line switches 41 – 43

executing 43 – 45

reflection and 45 – 46

buildscript 378

by keyword 282, **526**

by memoized call 479

bytecode 101 – **113**

concurrency through 149 – **168**

deadlock resolved, revisited 162 – **166**

deadlock revisited 160 – **162**

Lost Update 151 – **153**

synchronization in bytecode 153 – **157**

synchronized methods 157 – **158**

unsynchronized reads 158 – **160**

volatile access 166 – **168**

disassembling classes 101 – **103**

Kotlin and 551 – **553**

opcodes (operation codes)

arithmetic 107

execution flow control 108 – **109**

invocation 109 – **112**

load and store 106 – **107**

overview 105 – **106**

platform operation 112

shortcut forms of 112 – **113**

recursion and 507 – 509, 522, 524

runtime environment 103 – 105

C

C class 632

-c switch 101

C1 (client compiler) 235

C2 (server compiler) 235

cache misses 222 – 224

cached thread pool 205

call stack 104

call target 589

call() method 201, 268

Callable interface 201, 255, 584

CallSite object 589

cancel function 293

cancel() method 197

canonical constructor 63

capacity 211

CAS (Compare-and-Swap) operation 171

Chiusano, Paul 516

class files 82 – 86

bytecode 101 – 113

disassembling classes 101 – 103

opcodes (operation codes) 105 – 113

runtime environment 103 – 105

class loading and linking 83 – 85

initialization 85

preparation 85

resolution 85

verification 84 – 85

class objects 85 – 86

Classloader class 86 – 95

custom class loading 88 – 95

modules and class loading 95

examining 95 – 100

constant pool 98 – 100

internal form for method signatures 96 – 98

javap 96

reflection 113 – 118

combining class loading and 116 – 117

overview 114 – 116

problems with 117 – 118

class keyword 281

class loading 82

Class object 83, 85 – 86, 129, 573

Class type 99

Class<?> object 573

Class<Integer> type 117

classes, Kotlin 281 – 287

ClassFileParser::parseClassFile() method 92

ClassLoader class 86 – 95

custom class loading 88 – 95

dependency injection framework example 93 – 94

exceptions, class loading 89 – 91

first custom class loader 91 – 93

instrumenting class loader example 94 – 95

modules and class loading 95

clean lifecycle 351

-client switch 235

clj command 302

Clojure 260, 302

brackets 308 – 309

concurrent Clojure 557 – 570

agents 569 – 570

futures and pcalls 563 – 565

persistent data structures 557 – 563

software transactional memory 565 – 569

functional programming and closures 323 – 325

getting started with REPL 303 – 305

Gradle 376 – 377

Hello World in 302 – 303

interoperating between Java and 330 – 335

calling Java from Clojure 331 – 332

exploratory programming with REPL 334 – 335

Java type of Clojure values 332 – 333

nature of Clojure calls 332

using Clojure from Java 335

using Clojure proxies 333 – 334

Kotlin 271

macros 335 – 341

making mistakes 305 – 308

Maven 351 – 353

overview 300 – 302

property-based testing with 481 – 493

clojure.spec 483 – 486

clojure.spec and test.check 492 – 493

clojure.test 481 – 483

test.check 487 – 492

sequences 325 – 330

syntax and semantics 309 – 323

arithmetic, equality, and other operations 315 – 316

functions 316 – 319

lists, vectors, maps, and sets 312 – 315

loops 319 – 320

reader macros and dispatch 321 – 323

special forms 309 – 311

Testcontainers 467

Clojure FP 301, 325, 531 – 535

comprehensions 531 – 532

currying in Clojure 535

Kotlin FP 519 – 522

lazy sequences 532 – 534

overview 498 – 499

Clojure in Action (Rathore) 299

clojure.lang package 305

clojure.lang.ISeq interface 325

clojure.lang.Ref type 568

clojure.lang.Var type 333

clojure.spec 483 – 486, 492 – 493

clojure.test 481 – 483

clojure.test standard library 481

closures 325

Clojure 323 – 325

Java limitations as FP language 509 – 511

Kotlin FP 517 – 518

overview 500 – 501

Cluster object type 425

CMD Docker command 408 – 410, 413 – 414

CMT (chip multithreading) 220

CNCF (Cloud Native Compute Foundation) 433

Cognitect Labs test-runner test runner 483

Cohn, Mike 438

collect() method 511, 643, 646

collection literals 17

collections

Java limitations as FP language 514 – 515

Kotlin 277 – 279

limits of 645 – 646

Collections class 181

collections factories (JEP 213) 17 – 18

Collections helper class 642

Collectors.toList() static method 644

com.company.project convention 37

com.oracle.graal.graal_enterprise module 38

command-line switches 41 – 43

communication overhead 122

compact constructors 67

Compact Profiles (Java 8) 47 – 49

compact record constructors 67 – 69

compact strings 596 – 597

compacting 228

compareAndSwapInt() method 606

compareAndSwapLong() method 606

compareAndSwapObject() method 606

compareTo() method 539, 585

compilation error 288

compilation logs 237 – 238

compile goal 355

compile phase 350

compileClasspath Gradle dependency configuration 383 – 384

compiled languages 252 – 253

compileOnly Gradle dependency configuration 384

compiler fictions 267 – 268

CompletableFuture class 198 – 200, 499, 544 – 547

CompletableFuture.supplyAsync() method 200

CompletableFuture<T> type 199

compose function 516

comprehensions, Clojure FP 531 – 532

Computer Architecture: A Quantitative Approach (Hennessey et al.) 220

concurrency

block-structured concurrency (pre-Java 5) 128 – 147

deadlocks 132 – 135

fully synchronized objects 131 – 132

immunitibility 144 – 147

state model for threads 130 – 131

synchronization and locks 128 – 130

synchronization, reasons for 135 – 137

thread states and methods 137 – 143

volatile keyword 137

design forces 124 – 128

forces conflicting, reasons for 127

live system 126

performance 126

reusability 126 – 127

safety and concurrent type safety 125 – 126

sources of overhead 127 – 128

Java Memory Model (JMM) 147 – 149

Kotlin 291 – 294

theory primer 120 – 124

Amdahl's law 121 – 122

hardware 121

Java threading model 123 – 124

lessons learned 124

Thread acquaintance assumption 120 – 121

through bytecode 149 – 168

deadlock resolved, revisited 162 – 166

deadlock revisited 160 – 162

Lost Update 151 – 153

synchronization in bytecode 153 – 157

synchronized methods 157 – 158

unsynchronized reads 158 – 160

volatile access 166 – 168

concurrency primitive 170

concurrent programming

concurrent Clojure 557 – 570

agents 569 – 570

futures and pcalls 563 – 565

persistent data structures 557 – 563

software transactional memory 565 – 569

F/J (Fork/Join) framework 538 – 544

example of 539 – 542

parallelizing problems for 542

work-stealing algorithms 543 – 544

FP (functional programming) and 544 – 549

CompletableFuture class 544 – 547

parallel streams 547 – 549

Kotlin coroutines 549 – 556

coroutine scoping and dispatching 554 – 556

how coroutines work 549 – 554

ConcurrentHashMap class 176 – 185

approaches to concurrent Dictionary 180 – 182

limitations of Dictionary class 179 – 180

simplified HashMap 176 – 179

using 182 – 185

concurrently typesafe 125

ConcurrentMap interface 184

condition objects 173 – 174

configuration parameters, garbage collection 232 – 233

conj method 557

cons method 557

consensus barrier 174

consistency 349 – 350

constant pool 98 – 100

CONSTANT_ prefix 99

constructor keyword 283

Consumer<> interface 471

container engines 403

container registry 409

containers

Docker 406 – 411

building Docker images 406 – 409

developing Java applications with 411 – 424

running Docker containers 409 – 411

gathering container logs 471

importance of for developers 402 – 406

bare metal machines 402

benefits of containers 404 – 405

container engines 403

containers 403 – 404

drawbacks of containers 405 – 406

host operating system or Type 1 hypervisor 402

Type 2 Hypervisors 402 – 403

VM (virtual machines) 403

Kubernetes 424 – 432

observability 432 – 435

performance in 434 – 435

controllers 424

convenience and conciseness, Kotlin 271 – 281

collections 277 – 279

equality 273

functions 273 – 277

if expressions 279 – 281

starting with less 271 – 272

variables 272 – 273

cooperative multitasking 293

copy method 519, 550

copy-on-write semantics 185

CopyOnWriteArrayList class 185 – 188

coroutines 291, 549

overview 549 – 554

scoping and dispatching 554 – 556

CoroutineScope instance 555

cost, achieving higher performance 215 – 216

count method 455, 557

countDown() method 174

CountDownLatch 174 – 175

COWIterator object 186

create() factory method 335

critical section 129

curried function 502

currying

in Clojure 535

Java limitations as FP language 513

Kotlin FP 518

overview 502

custom class loading 88 – 95

dependency injection framework example 93 – 94

exceptions, class loading 89 – 91

first custom class loader 91 – 93

instrumenting class loader example 94 – 95

D

DAG (directed acyclic graph) 31

DAO (data access objects) 473

data class construct 519

data classes, Kotlin 286 – 287

data parallelism 547

Data Science Bookcamp (Apeltsin) 212

datatypes 341

DaySupplier type 512

deadlocks 132 – 135, 160 – 166

debugging in Docker 421 – 423

deconstruction patterns 610

def special form 301

default clause 60

default implementation 641

default implementation clash 642

default lifecycle 350

default line 59

default methods 641 – 642

defineAnonymousClass() method 607

defineClass() method 92

defineClass1() native method 92

defspec function 491

deftest function 482

degradation 211

delay function 550

delegated properties 479, 526

delegation 180

deoptimization, JIT (just-in-time) compilation 238

dependencies 346 – 349

dependency conflict 348 – 349

in Gradle 382 – 387

Maven 360 – 364

dependency configurations 382

dependency conflict 348

dependency injection framework example 93 – 94

dependency resolution 361

deploy phase 351

Deployment object type 425

deprecated thread methods 143

describe method 480

design forces, concurrency 124 – 128

conflicting, reasons for 127

live system 126

performance 126

reusability 126 – 127

safety and concurrent type safety 125 – 126

sources of overhead 127 – 128

destroy() method 143

destructuring 610

developers

build tools 345 – 350

automating tedious operations 346

ensuring consistency between developers 349 – 350

managing dependencies 346 – 349

containers, importance of 402 – 406

bare metal machines 402

benefits of containers 404 – 405

container engines 403

containers 403 – 404

drawbacks of containers 405 – 406

host operating system or Type 1 hypervisor 402

Type 2 Hypervisors 402 – 403

VM (virtual machines) 403

using language 264 – 265

DI (dependency injection) 81

Dictionary class 176

approaches to concurrent 180 – 182

limitations of 179 – 180

direct measurement 213 – 214

disjoint union types 73

dispatch reader macros, Clojure 321 – 323

dispatch, methods in JVM 572

dispatching, coroutines 554 – 556

distinguishability 222

Distributed traced observability pillar 433

div opcode 108

divide and conquer pattern 538

dload opcode 106

do-while loop 603

doc function 485

Docker 406 – 411

building Docker images 406 – 409

developing Java applications with 411 – 424

building image with Gradle 412 – 414

debugging in Docker 421 – 423

local development with Docker Compose 418 – 421

logging with Docker 423 – 424

ports and hosts 416 – 418

running build in Docker 414 – 416

selecting base image 411 – 412

running Docker containers 409 – 411

docker build command 406

docker commands 423

Docker Compose 418 – 421

docker cp command 422

docker pull command 409

docker push command 409

docker run command 409

- -dry-run flag 378

DSL (domain-specific language) 376, 378

duck typing 13

duck variable 13

dummy object 449 – 451

dup opcode 103, 106

dynamic compilation

monomorphic calls and 236 – 237

reasons for 234

dynamic typing 253 – 254

E

-e flag 410

eager evaluation 501, 646

Easy Cases Are Easy antipattern 542

echoserver container 426

effectively final variables 510

efficiency 211

EJBs (Enterprise Java Beans) 619

else case 281

else condition 337

else position 320

emergent property 208

end-to-end testing 440, 474 – 476

enterprise modules (JEP 320) 18 – 19

entry() method 18

entrySet() method 179

enum keyword 73

environment 410, 510

ephemeral port 417

equality

Clojure 315 – 316

Kotlin 273

equals() method 64

ergonomically 435

escape analysis 628

evaluation stack 104

EvilOrder class 70

exceptions 142 – 143

execute() method 543

executing, modular apps 43 – 45

execution control opcodes 108 – 109

<execution> steps 367

<executions> element 374

Executor argument 546

Executor interface 202

Executors class 202

Executors.newVirtualThreadPerTaskExecutor() factory method 625

ExecutorService interface 202

ExecutorService.invokeAll() helper method 564

exhaustiveness 70

ExpectedException rule 464

explain function 485

exploratory programming 303, 334 – 335

exporting keyword 36 – 37

exports keyword 33

EXPOSE ports Docker command 417

expressions 59

extends keyword 285

Extension objects 399

extensions property 399

ExternalResource class 461

F

F/J (Fork/Join) framework 538 – 544

example of 539 – 542

parallelizing problems for 542

work-stealing algorithms 543 – 544

factory method 144

failing tests, writing 443 – 445

failsafe plugin 459

fake object 449, 454 – 456

false values, Clojure 316

Feather, Michael 442

feed() method 115

Field object 85

Fieldref 100

files function 392

filter collection 516

filter functional technique 531

filter() method 511, 641 – 644

filter() pipeline 511

final keyword 146, 171, 498

final methods 578

final reference 504

final var combination 519

findClass() method 88

findConstructor() method 584

findVirtual() method 584

finite resources 402

fire-and-forget virtual threads 622

first-class functions 254, 268, 275

first() method 335

fixed-thread pool 204 – 205

fixtures 478

flatMap() method 546, 643

Flight Recorder 239 – 240

Float 100

for loop 278, 319, 604

forEach() method 643

forEach() operation 645

Foreign Function API 612 – 618

handling native memory in Panama 617 – 618

LibSPNG example 614 – 617

ForkJoinPool executor service 538

ForkJoinTask type 543

form 300

format() method 12

FP (functional programming)

Clojure 323 – 325

Clojure FP 531 – 535

comprehensions 531 – 532

currying in Clojure 535

lazy sequences 532 – 534

concepts of 498 – 502

closures 500 – 501

currying and partial application 502

higher-order functions 499 – 500

immutability 498 – 499

laziness 501

pure functions 498

recursion 500

concurrent programming and 544 – 549

CompletableFuture class 544 – 547

parallel streams 547 – 549

Java limitations as FP language 502 – 515

closures 509 – 511

currying and partial application 513

higher-order functions 505 – 506

Java type system and collections 514 – 515

laziness 511 – 512

mutability 503 – 505

pure functions 503

recursion 506 – 509

Kotlin FP 515 – 530

closures 517 – 518

currying and partial application 518

immutability 519 – 522

lazy evaluation 525 – 526

pure and higher-order functions 516 – 517

sequences 526 – 530

tail recursion 522 – 525

FROM Docker command 409, 414

full collections 228

fully synchronized objects 131 – 132

Function interface 644

Function objects 513

Function type 506

functional interface type 644

functional languages 254 – 255

Functional Programming in Kotlin (Vermeulen, Bjarnason, and Chiusano) 516

functionally orientated methods 642

functions 498

Clojure 316 – 319

Kotlin 273 – 277

Fusco, Mario 640

Future interface 197 – 200

futures, in Clojure 563 – 565

FutureTask class 200 – 201

fuzzing 489

FX (foreign exchange currency trading) 56

G

G1 collection 229 – 231

GA (general availability) 434

garbage collection 224 – 233

areas of memory 227

basics 225

configuration parameters 232 – 233

full collections 228

G1 229 – 231

mark and sweep 225 – 227

Parallel collector 231 – 232

safepoints 228

young collections 228

GAV (group, artifact, version) coordinates 352

gen function 493

generate function 488

generational GC 226

generators package 488

generic specialization 633

GenericContainer object 470

generics 633

get() method 145, 178, 197, 199, 313, 512

getAndAddInt() method 602

getAndAddLong() method 606

getAndIncrement() method 171

getAndSetInt() method 606

getAndSetLong() method 606

getAndSetObject() method 606

getCallerClass() 607

getClass() method 334

getDeclaredMethod() method 117

getfield opcode 106, 168

getInstance() method 237

getMethod() method 117

getstatic opcode 106

getSuperclass() method 85

goals 351

Goetz, Brian 168

Golden Hammer technique 117

goto opcode 108 – 109

Gough, James 246

GraalVM 261

Gradle 376 – 400

adding Kotlin 387 – 388

automating static analysis 389 – 390

building 379 – 381

building images with 412 – 414

customizing 397 – 400

creating custom plugins 399 – 400

custom tasks 397 – 399

dependencies in 382 – 387

installing 376 – 377

moving beyond Java 8 390

scripts 378

tasks 377 – 378

testing 388 – 389

using plugins 378 – **379**

using with modules 391 – **397**

JLink 393 – **397**

modular applications 392 – **393**

modular library 391 – **392**

work avoidance 381 – **382**

gradlew wrappers 377

gradlew.bat command 377

granularity 222

green threads 619

groom() method 575

Groovy

Kotlin vs. 378

overview 260

group constructs 478

group function 478

group lambdas 479

group method 480

guarded patterns 77

H

Happens-Before 147

hardware 121

hash() helper method 178

hashCode() method 60, 64

HashMap 111, 176 – 179

head-of-line blocking 19 – 20

headers, HTTP 21

heap 225

heap flattening 628

Hello World, in Clojure 302 – 303

hi symbol 310

hi var 310

hidden classes 607 – 608

higher-order functions

Java limitations as FP language 505 – 506

Kotlin FP 516 – 517

overview 499 – 500

homoiconic languages 336

host operating systems 402

hosts, Docker 416 – 418

HotSpot 235 – 236

C1 (client compiler) 235

C2 (server compiler) 235

real-time Java 235 – 236

HTTP/2 19 – 24

head-of-line blocking 19 – 20

HTTP header performance 21

in Java 11 22 – 24

other considerations 22

restricted connections 20 – 21

TLS 21 – 22

HttpClient type 22

HttpRequest type 22

HttpResponse.BodyHandler<T> interface 23

humongous region 230

I

IaaS (Infrastructure as a Service) 405

Idc opcode 106

identity-free references 631

if condition 508

if expressions 279 – 281

if opcodes 108

if statement 279, 448

IFn interface 307, 584

IFoo interface 641

- -illegal-access command-line switch 42, 45

ILP (instruction-level parallelism) 220

images

building Docker 406 – 409

building with Gradle 412 – 414

selecting base 411 – 412

imperative languages 254 – 255

implementation configuration 383

import statement 50

in keyword 280

incubating features 15 – 16

indexFor() helper method 178

infinite streams 646

init blocks, Kotlin 283

<init> method 103

initialization phase 85

inline types 628

inlining methods 234, 236

insert method 455

inst variable 100

Install phase 351

instanceof keyword 74 – 75, 630

instrumenting class loader example 94 – 95

Int type 288

Integer type 73, 100, 563, 626

integration testing 466 – 476

end-to-end testing with Selenium example 474 – 476

gathering container logs 471

installing testcontainers 467

Postgres, example with 472 – 473

Redis, example with 467 – 471

interface methods 575 – 576

InterfaceMethodref 100

internals

encapsulating 30

invokedynamic 589 – 593

method handles 583 – 589

looking up 585 – 586

MethodHandle 583 – 584

MethodType 584 – 585

reflection vs. proxies vs. 586 – 589

method invocation 571 – 578

final methods 578

interface methods 575 – 576

special methods 576 – 577

virtual methods 572 – 575

protecting 32 – 33

reflection internals 578 – 583

small internal changes 593 – 600

compact strings 596 – 597

nestmates 597 – 600

string concatenation 593 – 595

Unsafe

overview 600 – 604

replacing with supported APIs 604 – 608

interoperatability with Java

Kotlin 294 – 297

languages 263 – 264

with Clojure 330 – 335

calling Java from Clojure 331 – 332

exploratory programming with REPL 334 – 335

Java type of Clojure values 332 – 333

nature of Clojure calls 332

using Clojure from Java 335

using Clojure proxies 333 – 334

interpreted languages 252 – 253

interrupting threads 140 – 142

Introduction to Algorithms (Cormen et al.) 128

invocation opcodes 109 – 112

invoke function 275

invoke methods 584

invoke opcodes 102, 105, 110, 158

invoke() method 115, 307, 543, 578

InvokeDynamic 100

invokedynamic 64, 589 – 593

invokedynamic opcode 109, 267, 588

invokeExact() method 586

invokeinterface opcode 109, 111

invokespecial opcode 102 – 103, 109, 578

invokestatic opcode 109, 507

invokeSuspend method 552

invokevirtual opcode 103, 109, 572, 574

invoking methods 571 – 578

final methods 578

interface methods 575 – 576

special methods 576 – 577

virtual methods 572 – 575

is operator 290

isDone() method 197, 199

ISeq interface 328

isInterrupted() method 141

isthmus 612

it method 480

iterator() method 186, 334

J

jar tool 51

JARs, multirelease 49 – 52

building, example of 49 – 52

in Maven 366 – 369

Java

changes in Java 11 17 – 25

collections factories (JEP 213) 17 – 18

HTTP/2 (Java 11) 19 – 24

removal of enterprise modules (JEP 320) 18 – 19

single-file source-code programs (JEP 330) 24 – 25

Clojure interoperatability with 330 – 335

calling Java from Clojure 331 – 332

exploratory programming with REPL 334 – 335

Java type of Clojure values 332 – 333

nature of Clojure calls 332

using Clojure from Java 335

using Clojure proxies 333 – 334

Docker, developing applications with 411 – 424

building image with Gradle 412 – 414

debugging in Docker 421 – 423

local development with Docker Compose 418 – 421

logging with Docker 423 – 424

ports and hosts 416 – 418

running build in Docker 414 – 416

selecting base image 411 – 412

enhanced type inference (var keyword) 9 – 13

for free 637 – 638

language and platform features 13 – 16

changing language 14 – 15

defined 4 – 6

incubating and preview features 15 – 16

JSRs and JEPs 15

syntactic sugar 14

limitations as FP language 502 – 515

closures 509 – 511

currying and partial application 513

higher-order functions 505 – 506

Java type system and collections 514 – 515

laziness 511 – 512

mutability 503 – 505

pure functions 503

recursion 506 – 509

new Java release model 6 – 9

paid support options 639

Project Amber 610 – 612

Project Loom 618 – 625

programming with virtual threads 623 – 625

release of 625

Thread builder class 622 – 623

virtual threads 621 – 622

Project Panama 612 – 618

Project Valhalla 626 – 633

changing language model 630

consequences of value objects 630 – 633

generics 633

rigid syntax of 258 – 259

threading model 123 – 124

Java 8

backward compatibility 640 – 641

Compact Profiles 47 – 49

default methods 641 – 642

Gradle and 390

handling primitives 646 – 647

infinite streams 646

limits of collections 645 – 646

Maven and 365 – 366

parallel operations 648

streams 642 – 645

Java 11 17 – 25

collections factories (JEP 213) 17 – 18

HTTP/2 (Java 11) 19 – 24

removal of enterprise modules (JEP 320) 18 – 19

single-file source-code programs (JEP 330) 24 – 25

Java 17

instanceof operator 74 – 75

Pattern Matching and preview features 75 – 77

Records 60 – 69

compact record constructors 67 – 69

nominal typing 66 – 67

sealed types 69 – 74

Switch Expressions 57 – 60

Text Blocks 55 – 57

Java 18 633 – 634

java agent 94

java command 354, 396

Java Concurrency in Practice (Goetz) 168

Java EE (Java Enterprise Edition) 18

Java language 4

Java Module System, The (Parlog) 116

Java modules

architecting for modules 46 – 52

Java 8 Compact Profiles 47 – 49

multirelease JARs 49 – 52

split packages 47

basic syntax 34 – 37

exporting and requiring 36 – 37

transitivity 37

building first modular app 40 – 46

command-line switches 41 – 43

executing 43 – 45

reflection and 45 – 46

loading 37 – 40

application modules 39

automatic modules 39 – 40

platform modules 38 – 39

unnamed module 40

overview 27 – 34

module graph 31 – 32

new access control semantics 33 – 34

Project Jigsaw 28 – 31

protecting internals 32 – 33

Java performance tuning 220 – 224

cache misses 222 – 224

role of time in 221 – 222

accuracy 221 – 222

granularity 222

measurements 222

network-distributed timing 222

precision 221

java process 410

Java SE (Java Standard Edition) 18, 637 – 638

Java SE 8 638

Java SE 11 638

Java SE 17 (LTS) 638 – 639

Java Streams address 646

java-library plugin 380

java.base module 31, 581

java.io package 39

java.lang package 39

java.lang: IdentityObject interface 630

java.lang.Enum library type 70

java.lang.management package 49

java.lang.Record class 64

java.lang.reflect package 115

java.lang.reflect.Method class 115, 578

java.lang.String object 310

java.lang.Thread class 137, 200

java.lang.Thread subclass 622

java.lang.Void type 117

java.library.path system property 614

java.net.http module 16, 41

java.se module 39

java.time API 499

java.util collection interfaces 277

java.util package 39, 334

java.util.concurrent package 169

java.util.concurrent.atomic package 170

java.util.concurrent.Flow publisher 23

java.util.function interfaces 511

javac compiler 506

javac source code compiler 11

javap 95 – 96

jcmd 240

JCP (Java Community Process) 15

JDK concurrency libraries

atomic classes 170 – 172

BlockingQueue pattern 188 – 197

using BlockingQueue APIs 194 – 195

using WorkUnit 196 – 197

building blocks for modern concurrent applications 169 – 170

ConcurrentHashMap class 176 – 185

approaches to concurrent Dictionary 180 – 182

limitations of Dictionary class 179 – 180

simplified HashMap 176 – 179

using 182 – 185

CopyOnWriteArrayList class 185 – 188

CountDownLatch 174 – 175

Future type 197 – 200

lock classes 172 – 174

tasks and execution 200 – 206

cached thread pool 205

Executor interface 202

fixed-thread pool 204 – 205

modeling tasks 200 – 201

single-threaded executor 203

STPE (ScheduledThreadPoolExecutor) 205 – 206

JDK Flight Recorder 238 – 246

Flight Recorder 239 – 240

Mission Control 240 – 246

jdk.attach module 43

jdk.incubator namespace 16

jdk.incubator.foreign module 613

jdk.internal.jvmstat module 43

jdk.unsupported module 600

JEPs (JDK Enhancement Proposal) 15

jextract tool 613

JIMAGE format 28

jimage tool 29

JIT (just-in-time) compilation 233 – 238

deoptimization 238

dynamic compilation

monomorphic calls and 236 – 237

reasons for 234

HotSpot 235 – 236

C1 (client compiler) 235

C2 (server compiler) 235

real-time Java 235 – 236

inlining methods 236

reading compilation logs 237 – 238

jlink tool 29, 52, 393 – 397

JLS (Java Language Specification) 4, 147

JMC (JDK Mission Control) 240, 423

JMM (Java Memory Model) 120, 147 – 149, 601

jmod command 36

JMOD format 28

JNI (Java Native Interface) 15, 612

join() call 139

Joy of Clojure, The (Fogus and Houser) 299

JPMS (Java Platform Modules) 26

JRE (Java runtime library) 348

JRuby language 255

jshell interactive environment 12, 623

jsr opcode (deprecated) 108

JSRs (Java Specification Request) 15

JUnit 4 and 5 459 – 464

JUnit in Action (Tudose) 443

junit-vintage-engine dependency 460

JVM (Java's virtual machine) 30 – 31, 571

JVM_DefineClass() native method 83

JVM_DefineClassWithSource() C function 92

JVMTI (JVM Tool Interface) 213, 607

Jython language 255

K

K8s (Kubernetes) 424

key-fn function 318

keying function 318

keys function 485

keySet() method 179

klass 573

Knuth, Donald E. 216

Kotlin 259 – 260

concurrency 291 – 294

convenience and conciseness 271 – 281

collections 277 – 279

equality 273

functions 273 – 277

if expressions 279 – 281

starting with less 271 – 272

variables 272 – 273

coroutines 549 – 556

coroutine scoping and dispatching 554 – 556

how coroutines work 549 – 554

different view of classes and objects 281 – 287

Gradle adding 387 – 388

Groovy vs. 378

installing 271

Java interoperability 294 – 297

reasons for using 271

safety 287 – 291

Null safety 288 – 289

Smart casting 289 – 291

specification-style testing with 476 – 481

Kotlin FP 515 – 530

closures 517 – 518

currying and partial application 518

immutability 519 – 522

lazy evaluation 525 – 526

pure and higher-order functions 516 – 517

sequences 526 – 530

tail recursion 522 – 525

kotlin interactive shell 271

kotlin-coroutine-core library 291

kotlin-stdlib library 297, 388

kotlin.collections interfaces 277

kotlin.collections package 277

kotlin.collections.List interface 520

kotlin.collections.Map interface 520

kotlinc command-line compiler 271

kotlinx.collections.immutable library 521

Kt class 294

kubectl command 425 – 427

Kubernetes 424 – 432

L

L-type descriptors 632

lambda expressions 275, 591 – 593

LambdaMetafactory.metafactory() method 592

language features, Java 13 – 16

changing language 14 – 15

defined 4 – 6

incubating and preview features 15 – 16

instanceof operator 74 – 75

JSRs and JEPs 15

Pattern Matching and preview features 75 – 77

Records 60 – 69

compact record constructors 67 – 69

nominal typing 66 – 67

sealed types 69 – 74

Switch Expressions 57 – 60

syntactic sugar 14

Text Blocks 55 – 57

latches 174

lateinit pattern 479

latency 210

launch method 292, 552

layers 408

laziness

Clojure FP 532 – 534

Java limitations as FP language 511 – 512

Kotlin FP 525 – 526

overview 501

lazy evaluation 501, 646

lazy() function 525

Lazy<T> interface 525 – 526

LazySeq lazy implementation 333

LazyThreadSafetyMode enumeration 526

ldc (load constant) opcode 594

learning languages 264

library module 39

LibSPNG 614 – 617

lifecycle methods 202

LIMIT instance 70

LinkageError class 91

Lisp (Lots of Irritating Silly Parentheses) 308

list comprehensions 531

List interface 313, 641, 643

List object 641

- -list-modules flag 38

list-modules switch 41

listOf read-only helpers 520

lists, Clojure 312 – 315

live system, concurrency 126

load opcode 106 – 107

loadClass() method 88

loading and linking, class 83 – 85

automatic instrumentation via 214

combining with reflection 116 – 117

initialization 85

preparation 85

resolution 85

verification 84 – 85

loading Java modules 37 – 40

application modules 39

automatic modules 39 – 40

platform modules 38 – 39

unnamed module 40

local development, with Docker Compose 418 – 421

LocalDate class 512

lock classes 172 – 174

Lock objects 172 – 173

lock striping 183

log4j2 library 424

loggability guard 216

logging with Docker 423 – 424

Logs pillar 433

Long 100

lookupClass() expression 588

lookupswitch opcode 108

loops, Clojure 319 – 320

Lost Update 151 – 153

low-pause collector 229

low-risk project area 262 – 263

lreturn opcode 522

LVTI (Local Variable Type Inference) 10

M

M:1 threads 619

macro expansion time 336

macros, Clojure 304, 335 – 341

main method 87, 296, 354, 593

mainline development model 6

manifest, Maven 354

manual instrumentation 213

map collection 516

Map interface 111, 184

Map.Entry interface 177

map() method 314, 511, 528, 641 – 643, 647

maps, Clojure 312 – 315

mark and sweep 225 – 227

Maven 350 – 376

adding another language 355 – 357

authoring Maven plugins 372 – 376

build lifecycle 350 – 351

building 353 – 354

controlling manifest 354

dependency management 360 – 364

installing and POM (Project Object Model) 351 – 353

modules and 369 – 372

modular application 370 – 372

modular library 369 – 370

moving beyond Java 8 365 – 366

multirelease JARs in 366 – 369

reviewing 364 – 365

testing 357 – 360

Maven goal 373

maven-comiler-plugin 367

maven-failsafe-plugin 359

maven-publish plugin 392

maven-surefire-plugin 359

measurements, performance analysis 212 – 217

cost of achieving higher performance 215 – 216

dangers of premature optimization 216 – 217

how to take 213 – 214

automatic instrumentation via class loading 214

direct measurement 213 – 214

performance goals 214 – 215

time and 222

what to measure 212 – 213

when to stop 215

megamorphic 582

memoized 479

memory

areas of 227

handling native memory in Panama 617 – 618

latency hierarchy 219 – 220

software transactional memory 565 – 569

Memory API 612 – 618

handling native memory in Panama 617 – 618

LibSPNG example 614 – 617

MemoryAddress class 613

MemorySegment interface 613, 617

MergeSort algorithm 539

Meszaros, Gerard 449

metafactories 592

Method class 579

method handles 583 – 589

looking up 585 – 586

MethodHandle 583 – 584

MethodType 584 – 585

reflection vs. proxies vs. 586 – 589

Method Handles API 583

method invocation 571 – 578

final methods 578

interface methods 575 – 576

special methods 576 – 577

virtual methods 572 – 575

Method object 85, 115, 579

method signatures 96 – 98

MethodAccessor object 580

MethodHandle 100, 592

MethodHandles.Lookup type 589

MethodHandles.lookup() method 584

Methodref 100

methods 498

MethodType 100, 584 – 585, 589

Metrics pillar 433

minikube command 425

Mission Control 240 – 246

mistakes, in Clojure 305 – 308

mixed collection 230

mock object 449, 456 – 457

mock() method 457

mocking 457 – 458

modeling tasks 200 – 201

Callable interface 201

FutureTask class 201

Modern Java in Action, 2nd ed. (Urma, Fusco, Mycroft) 640

modular applications

Gradle 392 – 393

Maven and 370 – 372

modular Java runtime 28 – 29

modular library

Gradle 391 – 392

Maven and 369 – 370

module graph 31 – 32

module keyword 35

module name 36

module path 38

module resolution 95

module-info.java declarations 394

module-info.java module descriptor 614

- -module-path switch 41

ModuleDeclaration production 35

ModuleDescriptor type 116

ModuleDirective production 35

modules 27

class loading and 95

Gradle

JLink 393 – **397**

modular applications 392 – **393**

modular library 391 – **392**

Gradle and 391 – **397**

Maven and 369 – **372**

modular application 370 – **372**

modular library 369 – **370**

monitorenter opcode 112, **154**, **157**

monitorexit opcode 112, **154**, **157**

monomorphic calls 235 – **237**

Moore's law 217 – **219**

mul opcode 108

multimethods 341

multirelease JARs 49 – 52

building, example of 49 – 52

in Maven 366 – 369

multistage builds 414

mutability 503 – 505

mutable by default 301

mutable state 304

mvn dependency:tree command 362

Mycroft, Alan 640

N

- -name container-name parameter 421

NameAndType 100

named tuples 66

native methods 612

natural numbers 531

negative caching 90

nestmates 597, 600

network-distributed timing 222

new keyword 281

new name 112

new opcode 103

NEW thread state 130

newCondition() method 174

newFixedThreadPoolContext function 555

Newland, Chris 246

newWithPlatformReleaseN() method 641

next() method 335

nodes 558

nominal typing 66 – 67

Non-JVM languages 261 – 262

nondenotable types 12

nondeterministic pause 226 – 227

none function 279

nonrepeatable read 160

NOP (no-operation) 219

NoSuchMethodError exception 349

notify() method 630

NullPointerException (NPE) 288, 631

O

Object class 114, 630

object header 573

objectFieldOffset() method 602

objects

Kotlin 281 – **287**

Kubernetes 424

observability, containers 432 – **435**

ofEntries() method 18

offer() method 194

ofPlatform() method 622

ofVirtual() method 622

OO (object-oriented) languages 254

opcodes (operation codes)

arithmetic 107

execution flow control 108 – **109**

invocation 109 – **112**

load and store 106 – **107**

overview 105 – **106**

platform operation 112

shortcut forms of 112 – **113**

open keyword 35, **285**

open modules 45

OpenJDK 637 – **638**

opens keyword 35

OpenTelemetry 433 – **434**

operations

automating 346

Clojure 315 – 316

Optimizing Java (Evans, Gough, Newland) 246

Optional type 288

or function 484

Oracle JDK 637 – 638

Oracle OpenJDK Builds 637 – 638

orchestrators 424

orderly shutdown 203

OrderPartition type 68

org.apache.catalina.Executor class 206

org.beryx.jlink plugin 395

org.graalvm.js.scriptengine module 38

org.graalvm.sdk module 38

org.junit.jupiter.junit-jupiter-api dependency 459

org.junit.jupiter.junit-jupiter-engine dependency 459

org.testcontainers:testcontainers library 472

original languages, reimplementation vs. 255 – 256

OS (operating system) 619

OSR (on-stack replacement) 238

overhead, concurrency 127 – 128

P

-P switch 417

-p switch 577

Package phase 350

package-protected default 282

paid support options 639

Parallel collector 231 – 232

parallel operations 648

parallel streams 547 – 549

parallel() method 648

parent class loader 88

Parlog, Nicolai 116

partial application

Java limitations as FP language 513

Kotlin FP 518

overview 502

partial order 147

pass-by-value language 510

passing tests, writing 445

pattern matching 74 – 77, 290

pattern variables 75

pause goal 229

pcalls function 563 – 565

pending queue 193

performance

alternative JVM languages 266

concurrency 126

goals for 214 – 215

in containers 434 – 435

performance analysis

garbage collection 224 – 233

areas of memory 227

basics 225

configuration parameters 232 – 233

full collections 228

G1 229 – 231

mark and sweep 225 – 227

Parallel collector 231 – 232

safepoints 228

young collections 228

Java performance tuning 220 – 224

cache misses 222 – 224

role of time in 221 – 222

JDK Flight Recorder 238 – 246

Flight Recorder 239 – 240

Mission Control 240 – 246

JIT (just-in-time) compilation 233 – 238

deoptimization 238

dynamic compilation 234, 236 – 237

HotSpot 235 – 236

inlining methods 236

reading compilation logs 237 – 238

pragmatic approach to 212 – 217

cost of achieving higher performance 215 – 216

dangers of premature optimization 216 – 217

how to take measurements 213 – 214

performance goals 214 – 215

what to measure 212 – 213

when to stop 215

terminology 209 – 211

capacity 211

degradation 211

efficiency 211

latency 210

scalability 211

throughput 210

utilization 210

why to care about 217 – 220

memory latency hierarchy 219 – 220

Moore's law 217 – 219

PermGen (Permanent Generation) 231

permgen Java heap 572

permit | warn | deny setting 45

permits keyword 72

persistent data structures 521, 557 – 563

PersistentVector.Node inner class 559

phases 350

platform features, Java 13 – 16

changing language 14 – 15

defined 4 – 6

incubating and preview features 15 – 16

JSRs and JEPs 15

syntactic sugar 14

platform modules 38 – 39

platform operation opcodes 112

PlatformClassLoader 86

<plugin> element 354

plugins 351

authoring Maven 372 – 376

creating custom plugins 399 – 400

overview 378 – 379

plus function 555

Pod object type 425

Polish notation 308

poll() method 194

polyglot programming on JVM 256 – 262

contenders 260 – 262

GraalVM 261

Groovy 260

Non-JVM languages 261 – 262

Scala 260 – 261

non-Java language, reasons for using 257 – 259

up-and-coming languages 259 – 260

Clojure 260

Kotlin 259 – 260

POM (Project Object Model) 351 – 353

ports, Docker 416 – 418

Postgres 472 – 473

Postgres Testcontainer object 473

precision 221

Predicate interface 644

predicates 278

preemptive threads 624

premature optimization 216 – 217

preparation phase 85

preprocessor directives 336

preview features 15 – 16, 75 – 77

Price interface 451, 467

primitive classes 628

primitive specialization 647

primitive value types 632

primitives 646 – 647

ProcessHandle class 50

Project Amber 609 – 612

<project> element 354, 358

Project Jigsaw 28 – 31

encapsulating internals 30

JVM is modular now 30 – 31

modular Java runtime 28 – 29

Project Lambda 640

Project Loom 618 – 625

programming with virtual threads 623 – 625

release of 625

Thread builder class 622 – 623

virtual threads 621 – 622

Project Panama

Foreign Function and Memory API 612 – 618

handling native memory in Panama 617 – 618

LibSPNG example 614 – 617

overview 612

Project Valhalla 626 – 633

changing language model 630

consequences of value objects 630 – 633

generics 633

properties 487

<properties> element 354

property testing 263

property-based testing 263, 481 – 493

clojure.spec 483 – 486

clojure.spec and test.check 492 – 493

clojure.test 481 – 483

test.check 487 – 492

protocols 341

provides keyword 35

proxies

Clojure 333 – **334**

reflection vs. 586 – **589**

public static final variables 70

public visibility 284

pure functions

Java limitations as FP language 503

Kotlin FP 516 – **517**

overview 498

put() method 111, **178**

putAll() method 179

putfield opcode 106, **152**, **159**, **168**, **630**

putIfAbsent() method 184

putstatic opcode 102, **106**

pwd command 410

Q

Q-type descriptor 632

qualified exporting 39

query variable 56

Queue interface 188

quick-check function 490

R

RAII (Resource Acquisition Is Initialization) pattern 617

ranges 529

reader macros 321 – 323

real-time Java 235 – 236

rebinding 304

receiver object 115

Records 60 – 69

compact record constructors 67 – 69

nominal typing 66 – 67

record class 63

record components 63

record naming style 145

recurrence relation 534

recursion

Java limitations as FP language 506 – 509

Kotlin 522 – 526

overview 500

Redis 419, 467 – 471

redis key 419

redis object 471

reduce() method 643, 645

ReentrantLock implementation 172

ReentrantReadWriteLock implementation 172

refactoring tests 445 – 446

referential transparency 498

reflection 45 – 46, 113 – 118

combining class loading and 116 – 117

internals 578 – 583

overview 114 – 116

problems with 117 – 118

proxies vs. 586 – 589

reimplementation languages 255 – 256

- -release flag 51

remove (int index) list mutation method 514

remove() method 184

REPL (read-evaluate-print loop) 302

exploratory programming with 334 – 335

getting started with 303 – 305

replace() method 184

requires keyword 35 – 37

resolution phase 85

ResourceScope class 617

response object 23

restricted connections 20 – 21

restricted keywords 35

ret opcode (deprecated) 108

reusability 126 – 127

Rhino language 255

root module 40

RUN Docker command 407

run-tests function 482

run() method 102, 138, 200

Runnable interface 255

RUNNABLE thread state 130, 138

runtime environments for non-Java languages 266 – 267

runtime wiring 93

runtime-managed concurrency 121

runtimeClasspath Gradle dependency configuration 383 – 384

runtimeOnly Gradle dependency configuration 384

rust-bindgen tool 618

S

safepoints 228

safety

concurrent type safety 125 – 126

Kotlin 287 – 291

Null safety 288 – 289

Smart casting 289 – 291

SAM (single abstract method) 146

Scala 260 – 261

scalability 211

scalarization 628

scheduleAtFixedRate() method 205

ScheduledThreadPoolExecutor (STPE) 205 – 206

scheduleWithFixedDelay() method 205

Schwartzian transform 318

<scope> element 382

scopes 550

scoping, coroutines 554 – 556

scripts, Gradle 378

sealed interface 72

sealed keyword 73

sealed types 69, 71 – 74

secondary constructors 284

SEDA (Staged Event Driven Architecture) approach 621

SegmentAllocator interface 613, 617

Selenium 474 – 476

semantic versioning 349

semantics. *See* syntax and semantics

send method 23

sendAsync method 23

seq (sequence) 325

seq method 557

sequence lambda 530

Sequence<T> interface 526

sequenceOf() function 528

sequences 527

Clojure 325 – 330

Kotlin FP 526 – 530

Server JRE 28

-server switch 235

service abstraction 430

service bean 94

Service Kubernetes object type 425

Set interface 179, 643

setAccessible() method 45, 117

sets, Clojure 312 – 315

shallow immutability 147

shared mutable state 544

shortcut forms 112 – 113

shrinking 491

shutdown flag 167

shutdown() method 195

side-effect free 498, 644

single-file source-code programs (JEP 330) 24 – 25

single-threaded executor 203

site lifecycle 351

size() method 644

Skiena, Steven 128

small internal changes 593 – 600

compact strings 596 – 597

nestmates 597 – 600

string concatenation 593 – 595

smart casting 289

software transactional memory 565 – 569

sort() method 641

- -source flag 24

special forms, Clojure 309 – 311

special methods 576 – 577

specification-style testing 476 – 481

specifications 476

Spek 476 – 481

split packages 47

SREs (software reliability engineers) 433

SSD (solid-state drive) 219

stack 225

start() method 138, 622

startVirtualThread() static method 622

state machine 550

state-dependent update 137

static analysis 389 – 390

static methods 287

static synchronized method 129

static typing 253 – 254

STDERR output stream 424

STDOUT output stream 424

Steele, Guy L. 226

stop() method 166, 293

store opcode 106

Stream abstraction 640, 642

Stream interface 511, 642 – 643

Stream object 643

stream() method 511, 643

streams 642 – 646

String constructors 595

String representations 597

String type 73, 273, 288, 589, 593

StringBuilder object 216, 594

StringConcatFactory class 595

strings

compact 596 – 597

concatenation 593 – 595

StringSeq class 335

struct array 629

structural sharing 557

structural typing 13, 66

stub object 449, 451 – 453

STW (Stop-the-World) 226

sub opcode 108

submit() method 543

Succeeding with Agile (Cohn) 438

sudo ip addr show command 418

sum type 73

sun.misc.Unsafe class 170, 600

sun.net package 33

supplyAsync() factory method 546

surefire plugin 459

suspend functions 293, **530**, **550**

suspend keyword 293, **550**

suspend() method 143

Sutter, Herb 218

switch expressions 57 – **60**

switch statements 57

Symbol class 305

synchronization

in bytecode 153 – **157**

locks and 128 – **130**

reasons for 135 – **137**

synchronization-based concurrency 119

synchronized keyword 120, **157**

synchronized methods 150, **157** – **158**

synchronizedMap() method 181

Synchronizes-With relationship 147

syntactic sugar 14

syntax and semantics

Clojure 309 – **323**

arithmetic, equality, and other operations 315 – **316**

functions 316 – **319**

lists, vectors, maps, and sets 312 – **315**

loops 319 – 320

reader macros and dispatch 321 – 323

special forms 309 – 311

Java modules 34 – 37

exporting and requiring 36 – 37

transitivity 37

synthetic access method 599

System.currentTimeMillis() method 110

System.getenv() method 411

T

tableswitch opcode 108

tail position 508

tail recursion 507, 522 – 525

tailrec keyword 524

tailrec non-tail-recursive function 525

take() method 194, 529

TaskManager object 167

tasks and execution 200 – 206

cached thread pool 205

Executor interface 202

fixed-thread pool 204 – 205

Gradle 377 – 378

modeling tasks 200 – 201

Callable interface 201

FutureTask class 201

single-threaded executor 203

STPE (ScheduledThreadPoolExecutor) 205 – 206

tasks.jar configuration 381

TDD (test-driven development) 441 – 448

multiple use cases example 446 – 448

overview 442 – 443

single use case example 443 – 448

refactoring test 445 – 446

writing failing test (red) 443 – 445

writing passing test (green) 445

terminal method 511

TERMINATED thread state 131

test calls 478

test doubles 449 – 458

dummy object 449 – 451

fake object 454 – 456

mock object 456 – 457

problems with mocking 457 – 458

stub object 451 – 453

test function 478

test lambdas 479

test method 480

Test phase 350

test support, alternative JVM languages 264

test task 477

test-compile phase 353

Test-Driven Development: By Example (Beck) 441, 443

test.check 487 – 493

test.check-compatible generators 492

testCompileClasspath Gradle dependency configuration 384

testCompileOnly Gradle dependency configuration 384

Testcontainers 466 – 476

end-to-end testing with Selenium example 474 – 476

gathering container logs 471

installing 467

Postgres, example with 472 – 473

Redis, example with 467 – 471

testImplementation Gradle dependency configuration 384, 389

testing

from JUnit 4 to 5 459 – 464

Gradle 388 – 389

how to 438 – 441

integration testing with Testcontainers 466 – 476

end-to-end testing with Selenium example 474 – 476

gathering container logs 471

installing testcontainers 467

Postgres, example with 472 – 473

Redis, example with 467 – 471

Maven 357 – 360

property-based testing with Clojure 481 – 493

clojure.spec 483 – 486

clojure.spec and test.check 492 – 493

clojure.test 481 – 483

test.check 487 – 492

reasons for 438

specification-style testing with Spek and Kotlin 476 – 481

TDD (test-driven development) 441 – 448

multiple use cases example 446 – 448

overview 442 – 443

single use case example 443 – 448

test doubles 449 – 458

dummy object 449 – 451

fake object 454 – 456

mock object 456 – 457

problems with mocking 457 – 458

stub object 451 – 453

testRuntimeClasspath Gradle dependency configuration 384

testRuntimeOnly Gradle dependency configuration 384

Text Blocks 55 – 57

then form 338

thenApply() method 546

theory primer, concurrency 120 – 124

Amdahl's law 121 – 122

hardware 121

Java threading model 123 – 124

lessons learned 124

Thread acquaintance assumption 120 – 121

Thread

acquaintance assumption 120 – 121

Thread builder class 622 – 623

Thread.Builder class 622

Thread.interrupted() method 141

Thread.State enum 130

Thread.stop() method 143

ThreadDeath exception 143

ThreadFactory instances 623

threading model 123 – 124

ThreadPoolExecutor class 203

threads

state model for 130 – 131

states and methods 137 – 143

deprecated thread methods 143

interrupting threads 140 – 142

working with exceptions and threads 142 – 143

throughput 210

tiered compilation 236

time and performance tuning 221 – 222

accuracy 221 – 222

granularity 222

measurements 222

network-distributed timing 222

precision 221

time variable 110

TIMED_WAITING thread state 131

timeslice 624

TLS, HTTP/2 (Java 11) 21 – 22

to keyword 35

ToIntFunction special function type 647

tooling, alternative JVM languages 264

toString() method 64, 585

touchEveryItem() function 223

transitive closure 85

transitive dependencies 347

transitive keyword 35

transitivity 37

transport-level update 19

true values, Clojure 316

try-with-resources 617

Tudose, Cătălin 443

Turing completeness 500

Type 1 Hypervisors 402

Type 2 Hypervisors 402 – 403

type descriptors 96

type erasure 154

type hint 272

type inference (var keyword) 9 – 13

type pattern 75

type system 514 – 515

U

unbounded executor 625

union type 73

unit tests 439

unnamed module 40

Unsafe

overview 600 – **604**

replacing with supported APIs 604 – **608**

hidden classes 607 – **608**

VarHandles 605 – **607**

UnsupportedClassVersionError error 90

unsynchronized reads 158 – **160**

URLCanonicalizer class 32 – **33**

Urma, Raoul-Gabriel 640

user namespace 310, **483**

userid integer 9

uses keyword 35

Utf8 99 – **100**

utilization 210

V

val declaration 519

valid? function 483

Validate phase 350

value classes 628, 630 – 633

value record 629

ValueObject interface 630

values 510

values() method 179

Var class, Clojure 305

var keyword 253, 272

var object, Clojure 302

var properties, Kotlin 290

VarHandles 605 – 607

variables, Kotlin 272 – 273

variadic functions 329 – 330

variant code 49

vectors, Clojure 312 – 315

verification phase 84 – 85

Verify phase 351

verifying behavior 457

Vermeulen, Marco 516

virtual methods 572 – 575

virtual threads

overview 621 – 622

programming with 623 – 625

VirtualMachineDescriptor class 43

VMs (virtual machines) 403

VMSpec (JVM Specification) 4

void methods 117, 332

volatile keyword 120, 137

Volatile Shutdown pattern 166 – 168

W

wait-seq lazy sequence 565

wait() method 630

when keyword 280

when() method 457

while loop 140, 193, 319

with keyword 35

withers 499

withfield opcode 631

work avoidance, Gradle 381 – 382

work-stealing algorithms 543 – 544

Working Effectively with Legacy Code (Feather) 442

WorkUnit 196 – 197

wrappers 377

X

xUnit Test Patterns (Meszaros) 449

-XX:+UnlockCommercialFeatures option 240

Y

yield keyword 58, 554

young collections 228

Z

zombie, compilation logs and 238

Previous chapter

< Appendix B. Recap of streams in Java 8